
Model Evaluation, Model Selection and Hyperparameter Tuning

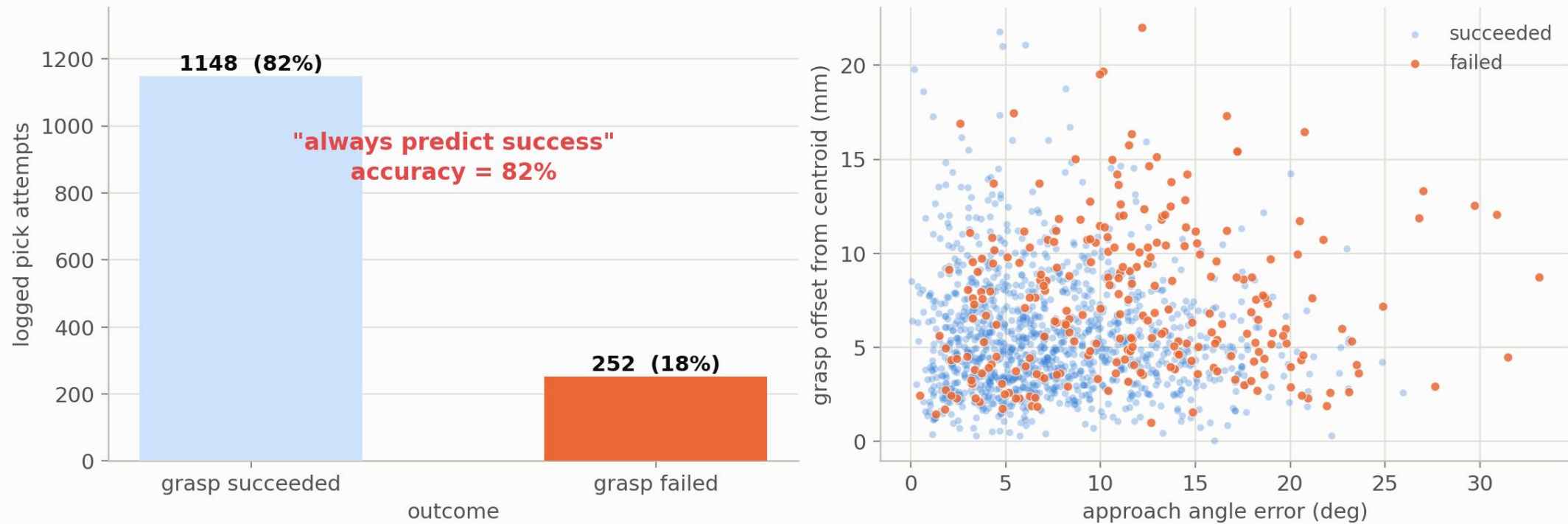
A worked case study: predicting robot grasp failures on a bin-picking cell

k-fold cross-validation · learning & validation curves · grid and random search
precision vs. recall · ROC curves and AUC

scikit-learn | companion notebook: `ML_evaluation_tuning_grasp.ipynb`

Will this grasp fail?

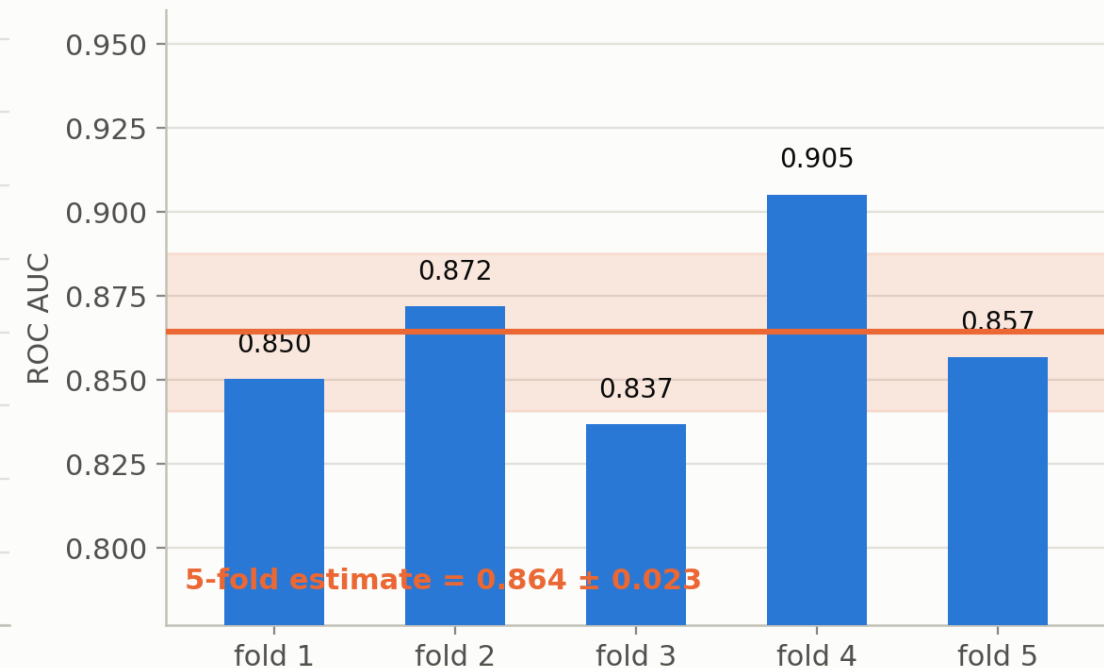
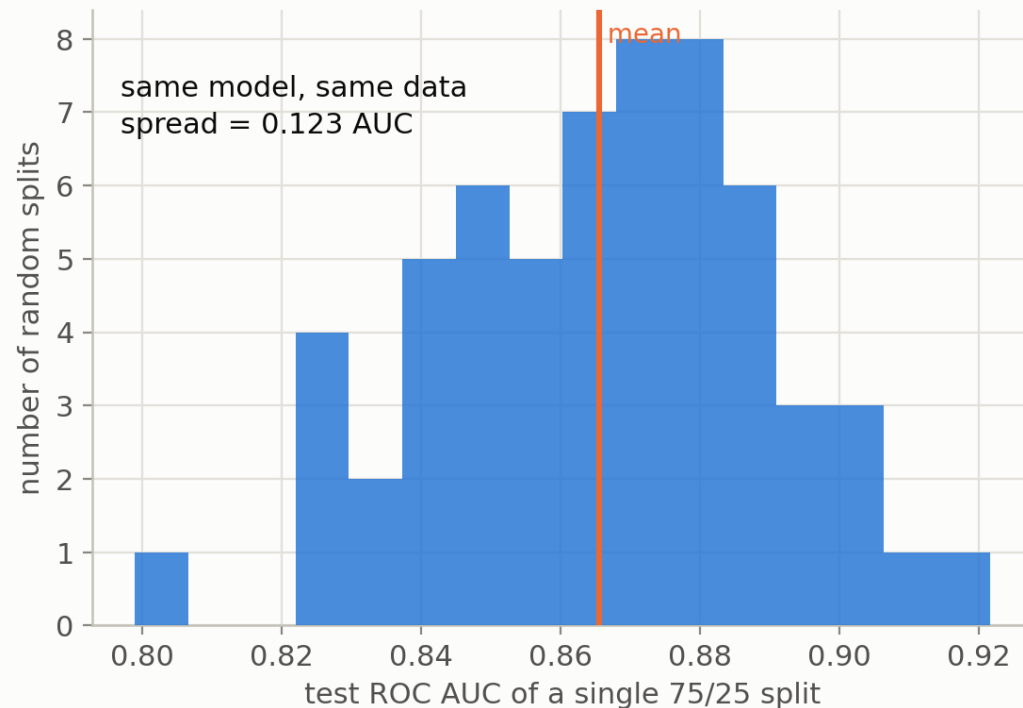
- 1400 logged pick attempts · 10 features known before the pick: part mass and width, friction, approach angle, grasp offset, grip force, clutter, F/T variance, pad wear.
- Target: grasp_failed. The positive class is the rare, expensive event — that is a deliberate choice, because precision and recall are always defined with respect to the positive class.



Only 18% of picks fail — so "always predict success" scores 82% accuracy and prevents exactly zero failures.

One train/test split gives one noisy number

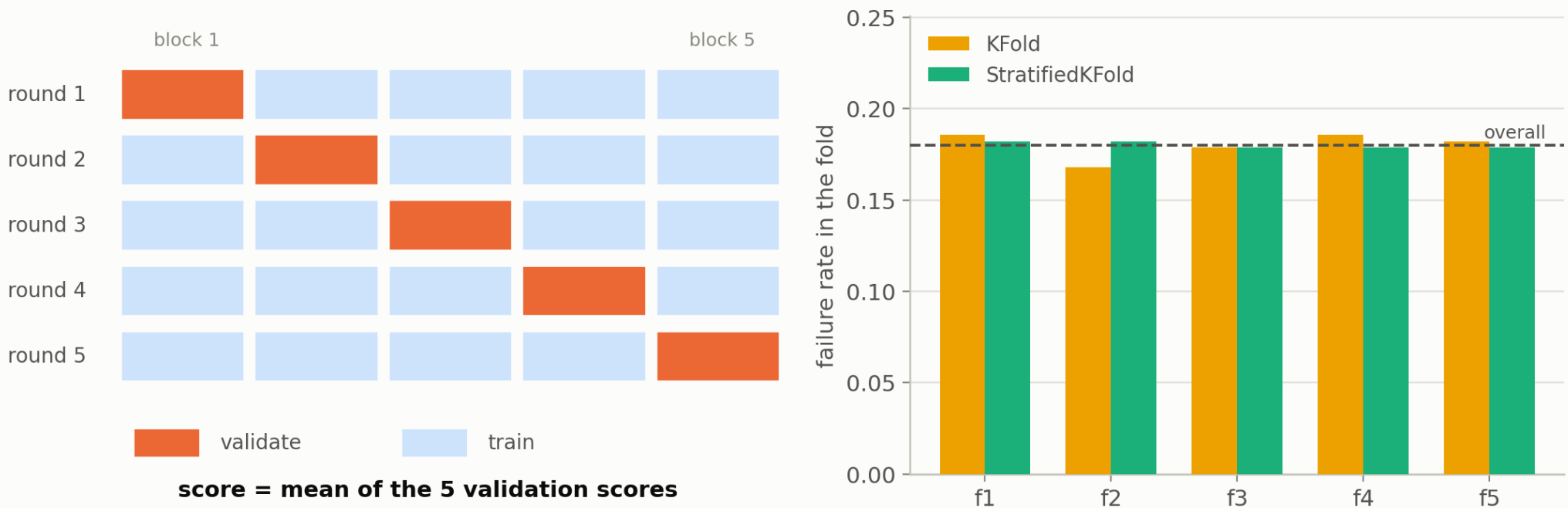
- Same model, same data, 60 different random 75/25 splits. Test ROC AUC ranges from 0.799 to 0.922.
- That 0.123 spread is larger than the gap between a good model and a mediocre one — so any comparison based on a single split is partly measuring luck.



The fix: average over several splits, and report the spread. 5-fold estimate = 0.864 ± 0.023 .

k-fold cross-validation

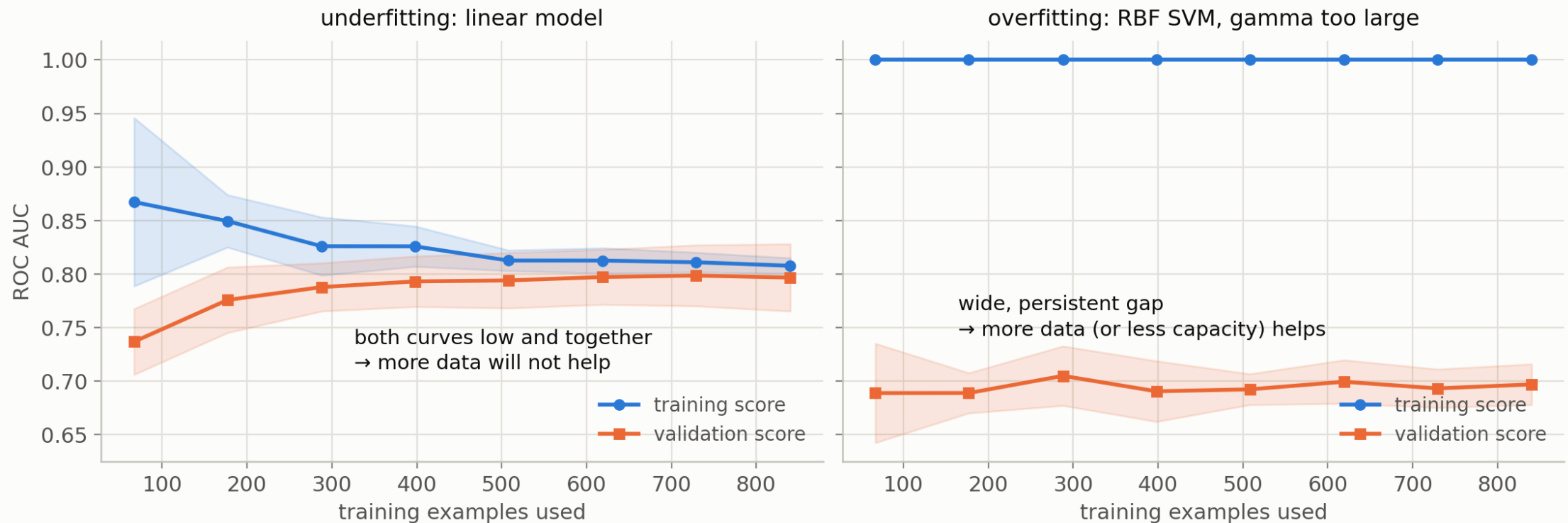
- Split the training data into k blocks. Round i trains on all blocks but block i and validates on block i . Every sample validates exactly once; the score is the mean of the k validation scores.
- $k = 5$ or 10 is the usual choice: larger k means more training data per round but k times the compute. Cost is k model fits per candidate model.



Always stratify for classification: StratifiedKFold pins every fold's failure rate to 0.180 (spread 0.004); plain KFold lets it drift (spread 0.018).

Learning curves: is it the data, or is it the model?

- A learning curve plots training and validation score against the number of training examples. It answers a question a single score cannot: would collecting more logged picks actually help?

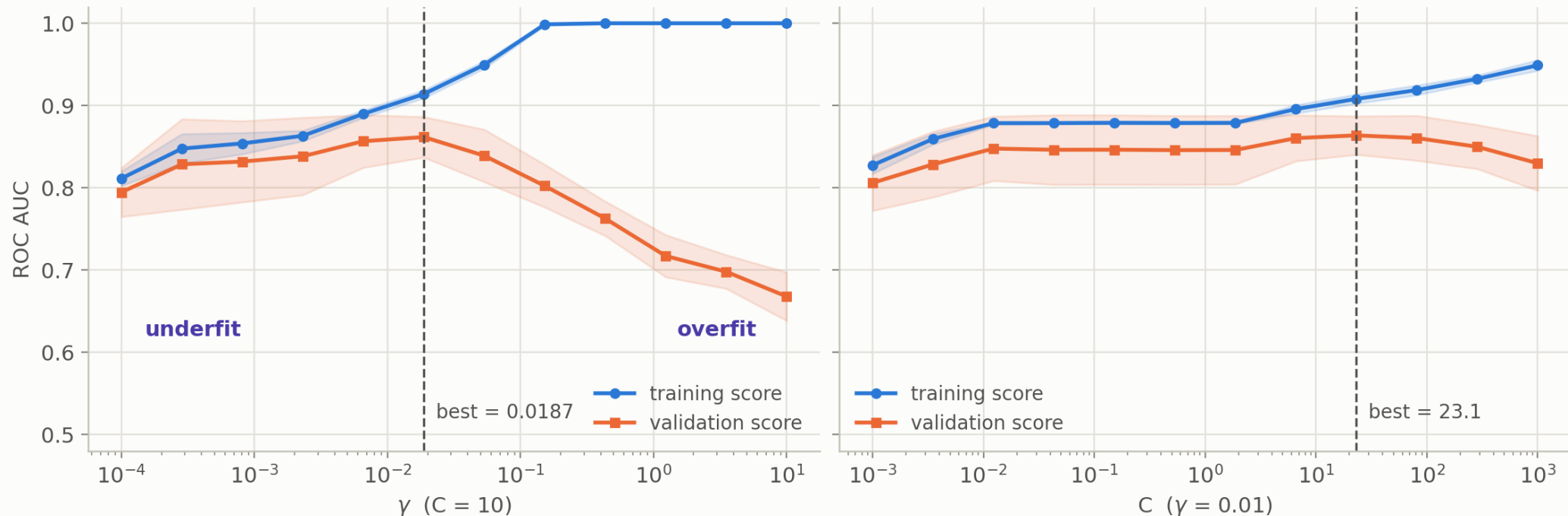


Underfitting (high bias) curves low and together, flat — validation stalls at 0.797. More data will not help; you need a better model or better features.

Overfitting (high variance) a persistent gap of 0.303 AUC — the model memorises. More data helps; so does turning the capacity down, which is free.

Validation curves: one hyperparameter at a time

- Same idea, but the x-axis is one hyperparameter with everything else fixed. Use it to find the interesting range and the shape of the trade-off – then search jointly.
- RBF SVM: γ is the reach of one training point in the kernel (small $\gamma \approx$ linear, large $\gamma \approx$ memorise); C is the price of a margin violation. Both are searched on a log scale.



Pick the peak of the validation curve, never the training curve. Best here: $\gamma \approx 0.019$, $C \approx 23$ – but one knob at a time is not the joint optimum.

Grid search: evaluate every combination

```
param_grid = {"svc__C": [0.1, 1, 10, 100],
              "svc__gamma": [0.001, 0.003, 0.01,
                             0.03, 0.1]}

grid = GridSearchCV(
    Pipeline([("scaler", StandardScaler()),
              ("svc", SVC(kernel="rbf"))]),
    param_grid, scoring="roc_auc",
    cv=StratifiedKFold(5, shuffle=True),
    n_jobs=-1, refit=True)
grid.fit(X_train, y_train)

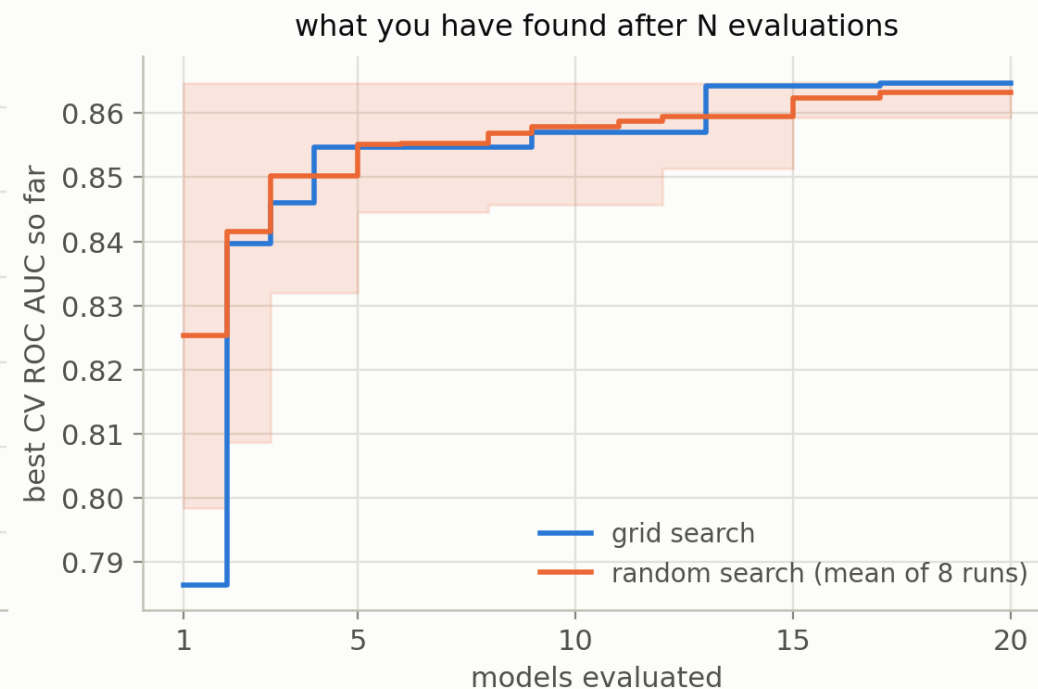
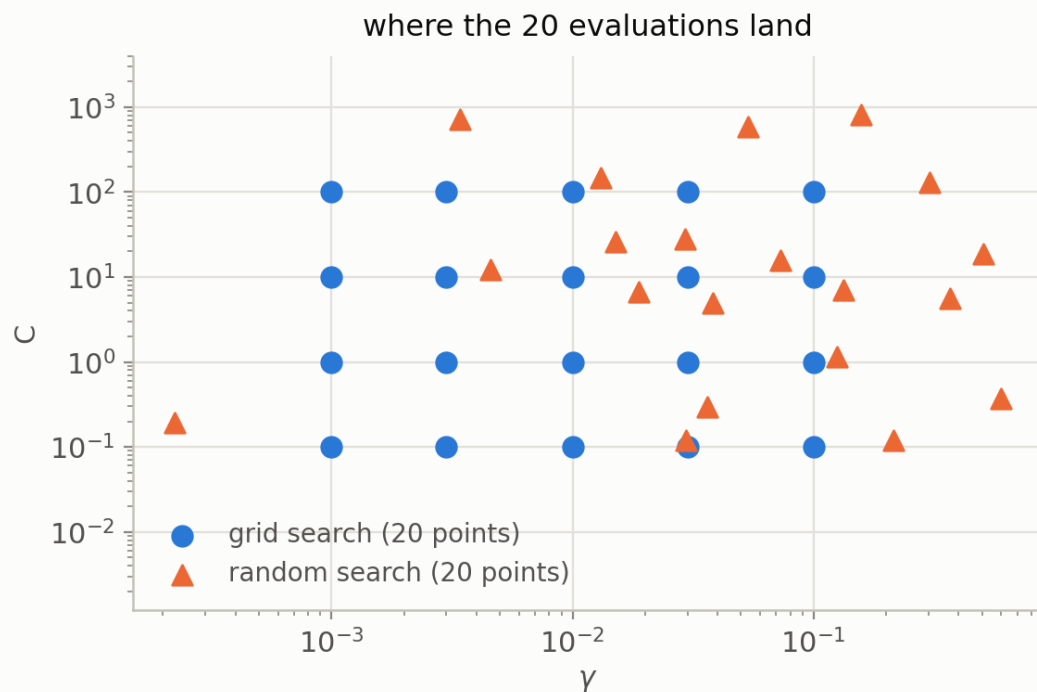
# grid.best_params_, grid.best_estimator_
```

- **$4 \times 5 = 20$ combinations $\times 5$ folds = 100 model fits.**
- Best: $C = 100$, $\gamma = 0.003 \rightarrow$ CV AUC 0.865 (linear baseline: 0.819).
- The `step__parameter` naming addresses a parameter inside a pipeline step — and the Pipeline is what keeps the scaler from seeing the validation fold.
- Cost grows as the product of the levels: add one more parameter with 5 values and you are at 500 fits.
- Check two things: is the winner on the edge of the grid (extend it), and how flat is the surface (if a dozen cells tie, prefer the simplest one).

Leakage rule: if it has a `.fit()` method — scaler, imputer, PCA, feature selector — it belongs inside the Pipeline, so it is refitted on each fold's training part only.

Random search: sample instead of enumerate

- RandomizedSearchCV draws `n_iter` combinations from distributions you specify — so you set the compute budget directly instead of inheriting it from the size of the grid. Use loguniform for scale parameters.



Why it wins as dimensions grow a 4-level grid tries only 4 distinct values of the knob that matters, however many points it evaluates. 20 random draws try 20 distinct values of every knob.

Rule of thumb ≤ 3 knobs with known ranges $\rightarrow$ grid. More knobs, or unknown ranges $\rightarrow$ random wide first, then a small grid inside the promising region. Same 100-fit budget here: grid 0.865, random 0.865.

Precision and recall: metrics that survive imbalance

- Fix a model, then look at what kind of mistakes it makes. Note what precision and recall have in common: neither counts true negatives — which is exactly why 1148 easy successes cannot inflate them.

actually succeeded	280 TN true negative	7 FP false alarm
	38 FN missed failure	25 TP caught failure
actually failed		
	predicted success	predicted failure

Precision

of the grasps we flagged, how many really would have failed

$$\frac{TP}{TP + FP} = 25 / (25 + 7) = 0.78$$

Recall (sensitivity)

of the grasps that really failed, how many did we flag

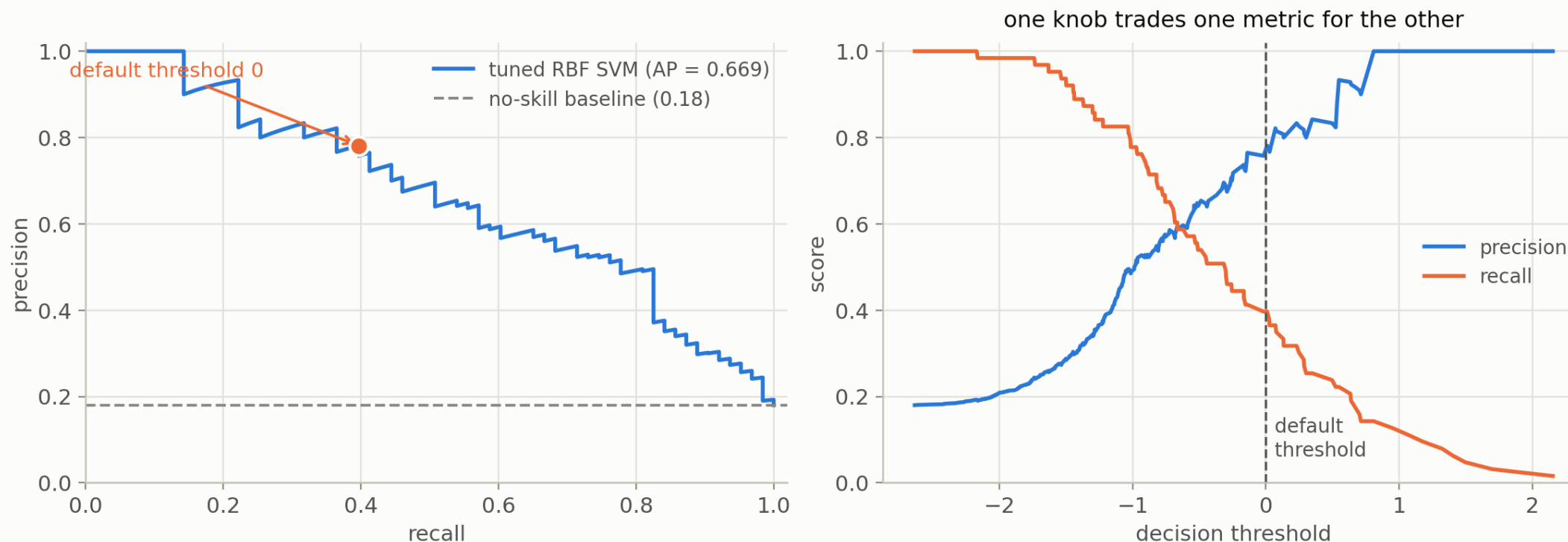
$$\frac{TP}{TP + FN} = 25 / (25 + 38) = 0.40$$

cost of FP: a needless re-plan cost of FN: a dropped part

F1 = harmonic mean of the two = 0.53. Which one you optimise is an engineering decision: here a dropped part costs far more than a needless re-plan, so you buy recall.

The threshold is a design decision

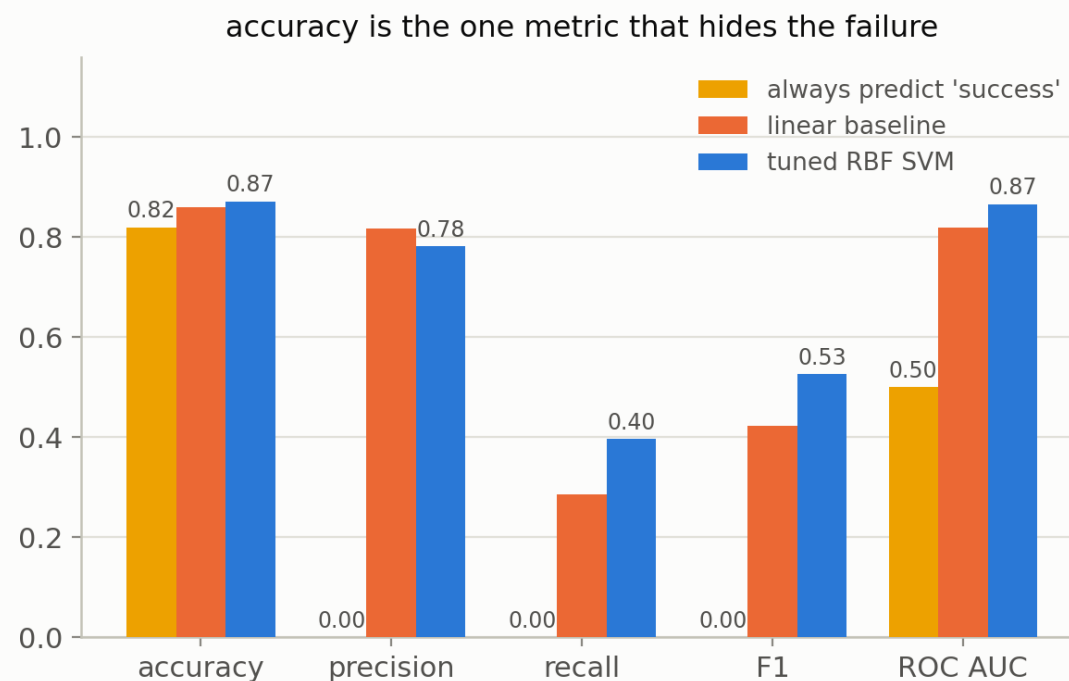
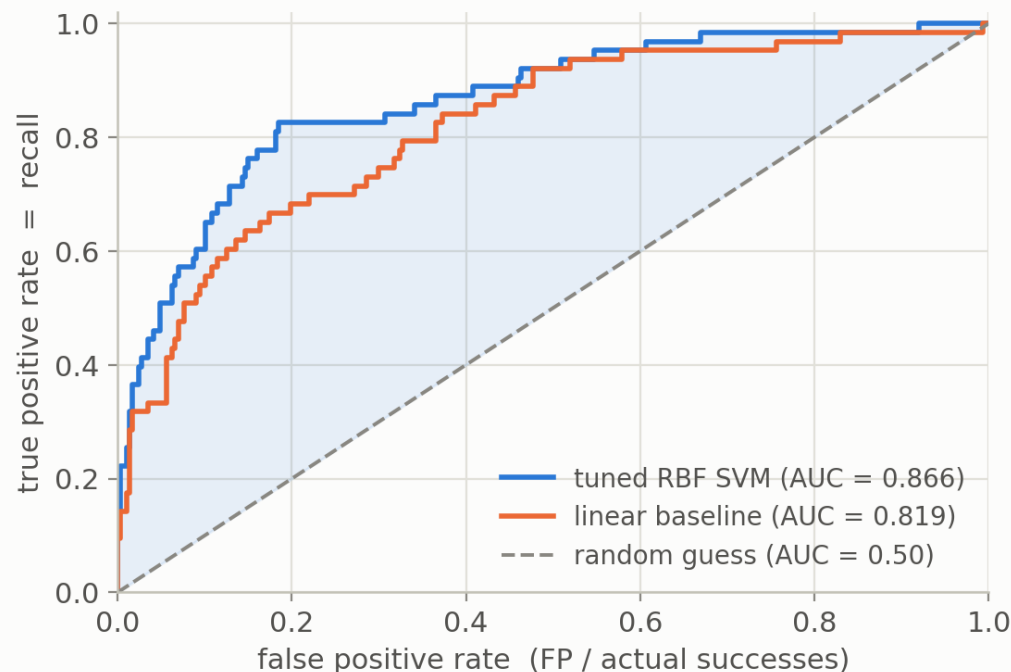
- `predict()` applies a fixed cut-off (score > 0 for an SVM, $p > 0.5$ for most others). Nothing forces you to keep it. Sliding it trades precision against recall along a curve.



Average precision (AP) summarises the whole curve: 0.669 here, against a no-skill baseline of 0.18 – the positive rate, not 0.5.

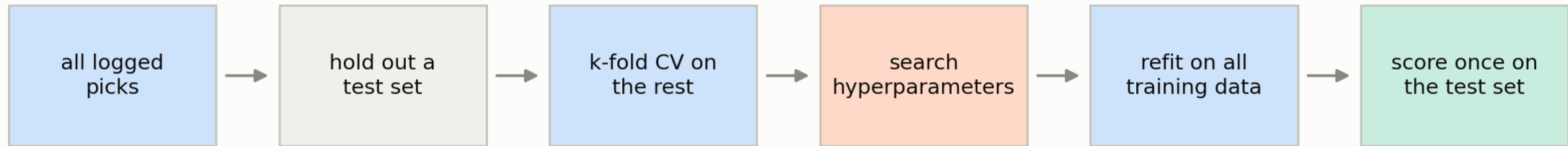
ROC curves and AUC

- Sweeping the same threshold, plot true positive rate (= recall) against false positive rate. AUC is the area underneath – and it equals the probability that a randomly chosen failing grasp is scored above a randomly chosen successful one.



AUC is threshold-free, so it is a good metric for choosing models and a poor one for deploying them – the cell still has to commit to a threshold.

What to take away



the test set is opened exactly once — every choice is made with CV scores

1. Open the test set exactly once, at the end. Every choice before that is made with CV scores.
2. Put every step with a `.fit()` inside the Pipeline, or preprocessing leaks across the fold boundary.
3. Stratify classification folds; split by group when rows are grouped.
4. Report mean $\pm$ std across folds — here 0.864 ± 0.023 , not a single lucky number.
5. Choose the metric from the cost of the mistakes. Accuracy is almost never that metric.
6. Diagnose before you tune: learning curve for data-vs-model, validation curve for the useful range.